21. Develop a C Program to Implement Worst Fit Algorithm of Memory Management

---

Code - Part 1 (Lines 1–35) — Memory Initialization & Display

```c
1  #include <stdio.h>
2
3  #define MAX_MEMORY 1000
4
5  int memory[MAX_MEMORY];
6
7  // Initialize memory
8  void initializeMemory() {
9      for (int i = 0; i < MAX_MEMORY; i++) {
10         memory[i] = -1;
11     }
12 }
13
14 // Display memory
15 void displayMemory() {
16     int i, j;
17
18     printf("\nMemory Status:\n");
19
20     for (i = 0; i < MAX_MEMORY; i++) {
21         if (memory[i] == -1) {
22             j = i;
23             while (j < MAX_MEMORY && memory[j] == -1) {
24                 j++;
25             }
26             printf("Free block: %d - %d\n", i, j - 1);
27             i = j - 1;
28         }
29     }
30 }
31
32 // Worst-Fit Allocation
33 void allocateMemory(int processId, int size) {
34     int maxSize = 0, maxStart = -1;
35
```

Code - Part 2 (Lines 36–67) — Worst-Fit Allocation Logic

```
36      int i = 0;
37
38      while (i < MAX_MEMORY) {
39          if (memory[i] == -1) {
40              int start = i;
41              int count = 0;
42
43              while (i < MAX_MEMORY && memory[i] == -1) {
44                  count++;
45                  i++;
46              }
47
48              if (count > maxSize) {
49                  maxSize = count;
50                  maxStart = start;
51              }
52          } else {
53              i++;
54          }
55      }
56
57      if (maxSize >= size) {
58          for (int i = maxStart; i < maxStart + size; i++) {
59              memory[i] = processId;
60          }
61
62          printf("Allocated block %d - %d to Process %d\n",
63                 maxStart, maxStart + size - 1, processId);
64      } else {
65          printf("Allocation failed for Process %d\n", processId);
66      }
67  }
68
```

Code - Part 3 (Lines 70–97) — Deallocation & Main Function

```
70 void deallocateMemory(int processId) {
71     for (int i = 0; i < MAX_MEMORY; i++) {
72         if (memory[i] == processId) {
73             memory[i] = -1;
74         }
75     }
76     printf("Memory released by Process %d\n", processId);
77 }
78
79 int main() {
80     initializeMemory();
81
82     displayMemory();
83
84     allocateMemory(1, 200);
85     displayMemory();
86
87     allocateMemory(2, 300);
88     displayMemory();
89
90     deallocateMemory(1);
91     displayMemory();
92
93     allocateMemory(3, 400);
94     displayMemory();
95
96     return 0;
97 }
```

Output

```
Memory Status:
Free block: 0 - 999
Allocated block 0 - 199 to Process 1

Memory Status:
Free block: 200 - 999
Allocated block 200 - 499 to Process 2

Memory Status:
Free block: 500 - 999
Memory released by Process 1

Memory Status:
Free block: 0 - 199
Free block: 500 - 999
Allocated block 500 - 899 to Process 3

Memory Status:
Free block: 0 - 199
Free block: 900 - 999


...Program finished with exit code 0
Press ENTER to exit console.
```